UNIVERSIDAD PARAGUAYO ALEMANA (UPA)

Carrera: Informática Empresarial

TRABAJO PRÁCTICO: ARQUITECTURA Y PLATAFORMA DIGITAL UNIVERSITARIA (UPA APP)

Metadato	Detalle Institucional
Institución:	Universidad Paraguayo Alemana (UPA) – San Lorenzo, Paraguay
Proyecto:	UPA App – Plataforma Digital y Campus Móvil Centralizado
Repositorio Oficial:	https://github.com/Pascal-Thommen/upa-app
Autores:	Pascal Thommen, Theresa Bauer, Natasha kolman
Version:	v2 (23. Oktober 2026)

Avances desde la última reunión

- Infraestructura:
  - Base de datos PostgreSQL conectada al backend.
  - API backend creada y documentada.
  - implementado en el servidor y conectado a Internet mediante netbird.

- Integraciones externas:
  - Feed de Instagram mediante SociableKIT.
  - Grupos de Google Workspace Admin.
  - Google Classroom mediante OAuth.

- Funciones principales:
  - ingresar mediante google OAuth
  - Gestión de noticias y eventos.
  - Sistema de roles y permisos para usuarios y administradores.

- Frontend provisional:
  - Dashboard para visualizar y comprobar que las APIs funcionan correctamente.
  - Prototipo inicial del área de administración para gestionar contenidos, eventos e imágenes.

- Próximos pasos:
  - Mejorar y validar el frontend.
  - Completar el área de administración.
  - Probar todas las integraciones con usuarios y datos reales.

android nativo
sql diagramo

1. Resumen Ejecutivo (Executive Summary)

El presente Trabajo Práctico documenta la concepción arquitectónica, la definición de alcance y el estado actual de desarrollo de la plataforma digital universitaria para la Universidad Paraguayo Alemana (UPA). Concebido desde la disciplina de la Informática Empresarial (Wirtschaftsinformatik) en la era de la inteligencia artificial y el desarrollo asistido, este documento no constituye un reporte de código manual ni un registro de tareas de programación, sino un informe técnico-estratégico de arquitectura ("del arquitecto hacia el propietario del sistema").

La iniciativa responde a una necesidad institucional crítica: la dispersión operativa de la información universitaria entre múltiples canales informales y plataformas desarticuladas (redes sociales, grupos de mensajería, aulas virtuales y portales administrativos). El objetivo central es consolidar un punto único de acceso (Single Source of Truth) para la comunidad estudiantil, integrando noticias oficiales, eventos institucionales, entregas académicas y alertas de emergencia en una solución móvil nativa, robusta y segura.

El proyecto ha completado con éxito su fase de infraestructura base, backend y arquitectura de integración:

- 1. Implementación de una base de datos relacional PostgreSQL gestionada mediante SQLAlchemy y Alembic.
- 2. Desarrollo de una API REST modular en FastAPI con documentación interactiva OpenAPI y contratos Pydantic v2.
- 3. Conexión segura al servidor mediante la red privada mallada NetBird.
- 4. Integración de servicios externos clave (Google OAuth 2.0 institucional @upa.edu.py, Google Workspace Admin API, Google Classroom API y feed visual de Instagram mediante SociableKIT).
- 5. Frontend móvil definido como una Aplicación Nativa Android (Kotlin, Jetpack Compose, Room/SQLite offline) y portal administrativo Web-Admin para la gestión editorial.

2. Definición del Problema y Objetivos de Negocio

2.1 Contexto y Diagnóstico Operativo

En el ecosistema universitario contemporáneo de la UPA, la comunicación institucional padece de fragmentación multicanal:

- **1. Dispersión y pérdida de información crítica:** Los comunicados urgentes (como suspensión de clases o cambios edilicios) compiten con mensajes informales en grupos de WhatsApp, diluyendo su carácter oficial y generando incertidumbre.
- **2. Duplicidad de esfuerzos en difusión:** El equipo de comunicación y marketing genera contenido institucional de alto impacto visual en Instagram, el cual no se refleja de forma centralizada ni estructurada en los canales académicos oficiales.
- **Falta de canal formal de confirmación:** Las actividades extracurriculares, charlas y talleres carecen de un mecanismo ágil y centralizado de registro de asistencia (RSVP), dificultando la planificación logística universitaria.

2.2 Objetivos Estratégicos del Proyecto

- **Centralización Institucional:** Establecer una única interfaz de referencia para la vida universitaria de directivos, docentes y alumnos.
- **Integración sin Redundancia:** Conectar los sistemas ya consolidados en la UPA sin intentar reemplazarlos innecesariamente ni duplicar costos operativos.
- **Seguridad y Control de Identidad:** Restringir el acceso exclusivamente a usuarios federados con cuentas institucionales @upa.edu.py.
- **Escalabilidad y Sostenibilidad:** Diseñar una arquitectura desacoplada, documentada y de bajo costo de mantenimiento para la universidad.

3. Delimitación del Alcance y Límites del Sistema (Scope Baseline)

3.1 Dentro del Alcance (In-Scope — Fase 1 / Versión Actual)

- **1. Autenticación y Seguridad Federada:**
    - Inicio de sesión único (SSO) mediante Google OAuth 2.0 restringido estrictamente al dominio @upa.edu.py.
    - Emisión y validación de tokens de sesión JWT (JSON Web Tokens) y endpoint seguro de dev-login para entornos de prueba controlados.
- **2. Módulo de Publicaciones Unificadas (Feed & Comunicados):**
    - Muro centralizado con soporte para tres formatos: noticias institucionales estándar, banners fijados de emergencia (Eilmeldungen) y publicaciones sociales.

    - Integración automatizada del feed oficial de Instagram de la UPA vía SociableKIT.
    - Carga de imágenes y afiches mediante endpoint dedicado /api/media/upload con almacenamiento persistente en /uploads.
- **3. Módulo de Agenda y Eventos (Calendario Centralizado):**
    - Visualización cronológica de eventos, charlas y talleres.
    - Sistema interactivo de confirmación de asistencia (RSVP) con estados confirmados (attending) y cancelados (declined).
    - Sincronización de fechas límites de entregas académicas desde Google Classroom API mediante tokens autorizados.
- **4. Segmentación y Control de Accesos (RBAC):**
    - Clasificación de usuarios con perfiles diferenciados (user y admin).
    - Consulta de grupos institucionales mediante Google Workspace Admin Directory API para segmentación de avisos por carrera y semestre.
    - Registro y auditoría inmutable de acciones administrativas (admin_audit_logs).
- **5. Panel de Gestión Web (Web-Admin CMS):**
    - Interfaz web responsiva para el equipo editorial y administrativo para redactar noticias, programar eventos, fijar avisos urgentes y cargar afiches.
- **6. Frontend Móvil Nativo Android:**
    - Aplicación nativa Android (API Level 34+, min SDK 26) desarrollada con Kotlin y Jetpack Compose.
    - Arquitectura MVVM (Model-View-ViewModel) con 3 vistas principales: Inicio (Feed y comunicados), Agenda (Calendario con deadlines) y Perfil.
    - Almacenamiento local mediante SQLite y Room para soporte de caché offline.
    - Integración de notificaciones push de alta prioridad para avisos de emergencia mediante Firebase Cloud Messaging (FCM).
- **7. Infraestructura de Despliegue Seguro:**
    - Contenerización de servicios mediante Docker y Docker Compose.
    - Conectividad segura al servidor de staging mediante red privada mallada NetBird (sin exposición de puertos vulnerables a internet público).

4. Arquitectura del Sistema y Fundamentación de Decisiones ("El Porqué")

4.1 Frontend Móvil: Adopción de Android Nativo y Descarte Explícito de PWA

- **Decisión:** Se adoptó una arquitectura 100% nativa en Android (Kotlin, Jetpack Compose, Room SQLite), descartando de manera deliberada la opción de una Progressive Web App (PWA).
- **Justificación técnica y estratégica (¿Por qué se descartó PWA?):**
    - *Confiabilidad de notificaciones push de emergencia (Eilmeldungen):* En una aplicación universitaria donde el aviso urgente (por ejemplo, suspensión de

clases por inclemencias climáticas) es crítico, las PWAs presentan limitaciones severas de entrega en segundo plano en sistemas móviles. La implementación nativa con Firebase Cloud Messaging (FCM) y DeviceTokens en Android garantiza la recepción instantánea de alertas prioritarias con el dispositivo bloqueado.
    - *Persistencia offline y caché robusto:* Mientras que el almacenamiento web en Service Workers / Cache API es susceptible a ser desalojado automáticamente por el sistema operativo cuando el almacenamiento escasea, Android Room con SQLite garantiza persistencia offline determinista y sincronización reactiva local sin degradación.
    - *Rendimiento y experiencia de usuario (UX):* Jetpack Compose proporciona renderizado declarativo nativo a 60/120 fps, transiciones fluidas, compatibilidad con temas del sistema (Light/Dark mode) y acceso a APIs de hardware sin la sobrecarga del motor del navegador.
    - *Foco estratégico del equipo:* En el marco del proyecto universitario, concentrarse en un frontend nativo Android robusto en lugar de lidiar con inconsistencias cross-browser permitió consolidar una arquitectura sólida y validable con el cuerpo estudiantil.

4.2 Backend: Adopción de FastAPI (Python) frente a Java / Spring Boot

- **Decisión:** Se seleccionó FastAPI (>=0.115.0) sobre Python con servidor asíncrono Uvicorn en lugar de un stack tradicional basado en Java / Spring Boot.
- **Justificación técnica y estratégica:**
    - *Velocidad de desarrollo (Time-to-Market) y sinergia con IA:* Dado el volumen proyectado de la comunidad universitaria (~500 estudiantes), los aspectos de concurrencia extrema quedan plenamente cubiertos por cualquier stack moderno. Por ende, el factor determinante fue maximizar la velocidad de prototipado e implementación. FastAPI ofrece un desarrollo altamente ágil con una sintaxis concisa que integra de manera óptima y nativa con herramientas de desarrollo asistido por Inteligencia Artificial, acelerando de forma sustancial los ciclos de iteración.
    -

    -
    - *Validación estricta de datos mediante Pydantic v2:* La integración nativa con Pydantic v2 garantiza la validación estricta de tipos de datos de entrada y salida mediante esquemas declarativos, previniendo incoherencias y filtrando cargas maliciosas antes de alcanzar la capa de negocio.

4.3 Base de Datos: Migración de SQLite a PostgreSQL con SQLAlchemy y Alembic

- **Decisión:** Implementación de PostgreSQL como motor relacional corporativo, gestionado mediante SQLAlchemy ORM (>=2.0.35) y migraciones automáticas con Alembic (>=1.13.3).
- **Justificación técnica y estratégica:**
    - *Concurrencia real de lectura y escritura:* SQLite implementa bloqueos a nivel de archivo para las operaciones de escritura. En un contexto universitario donde cientos de estudiantes confirman asistencia a eventos (RSVP) concurrentemente mientras los administradores publican avisos, SQLite generaría excepciones de contención (database is locked). PostgreSQL resuelve esto mediante MVCC (Multi-Version Concurrency Control), garantizando transacciones ACID simultáneas y sin bloqueos.
    - *Integridad referencial y seguridad de datos:* PostgreSQL asegura integridad mediante restricciones compuestas (como la clave única en (post_id, user_id) de event_registrations) y soporte para campos relacionales avanzados.
    - *Evolución de esquema controlada:* Con Alembic, cada cambio estructural en las tablas queda registrado en el historial de código del repositorio, permitiendo auditorías y migraciones reproducibles sin riesgo de pérdida de datos.

4.4 Integraciones Externas: Principio de Integración frente a Reinvención

- **Decisión:** Integración de SociableKIT (Instagram), Google Workspace Admin y Google Classroom.
- **Justificación técnica y estratégica:**
    - *Instagram vía SociableKIT:* El equipo de marketing de la UPA ya produce contenido visual activo y de calidad en Instagram. Programar un módulo de publicación social propio en la app universitaria resultaría en un costo redundante y duplicaría el trabajo editorial. Integrar el feed existente conecta la app directamente a la vida universitaria activa sin esfuerzo administrativo adicional.
    - *Google Workspace OAuth 2.0:* Toda la comunidad UPA cuenta con identidad digital institucional bajo @upa.edu.py. Desarrollar un sistema de registro propio con almacenamiento de contraseñas acarrearía responsabilidades graves de seguridad, costos de infraestructura SMTP y problemas operativos de recuperación de cuentas. OAuth delega la seguridad en la infraestructura

de Google y garantiza que solo miembros verificados de la UPA accedan al sistema.
    - *Google Classroom API:* Sincroniza directamente las tareas y fechas de entrega con el calendario de la app móvil, eliminando la necesidad de que los estudiantes consulten múltiples plataformas para conocer sus plazos académicos.

4.5 Infraestructura y Seguridad: Conectividad Zero-Trust mediante NetBird

- **Decisión:** Acceso seguro y publicación de servicios de la API y la web mediante la red de acceso seguro Zero-Trust de NetBird (disponible en https://upa.eu1.netbird.services/dashboard/).
- **Justificación técnica y estratégica:**
    - *Seguridad perimetral Zero-Trust y exposición pública controlada:* NetBird permite hacer accesibles públicamente la API REST y el portal web mediante un modelo de red overlay Zero-Trust. Esto garantiza una conectividad directa, cifrada de extremo a extremo y restringida granularmente por políticas de identidad para administrar y acceder a los servicios (https://upa.eu1.netbird.services/dashboard/).
    - Adicionalmente, esta solución resuelve de manera pragmática y segura los desafíos operacionales propios del entorno de desarrollo (como el alojamiento transitorio en un servidor local/casero sin IP pública fija) durante esta etapa del proyecto. Se destaca que esta infraestructura representa un entorno de staging robusto pero provisorio para la fase actual, manteniendo total flexibilidad para la posterior migración al hosting productivo definitivo que determine la universidad.

4.6 Modelo de Dominio: Fuente Única de Verdad (Unified Post Model)

- **Decisión:** Modelo relacional unificado Post para gestionar noticias, eventos y alertas urgentes.
- **Justificación técnica y estratégica:**
    - *Flexibilidad editorial y reducción de redundancia:* En la comunicación universitaria, los eventos son a menudo también noticias y pueden requerir ser destacados como avisos prioritarios (is_news, is_event, is_pinned). Mantener entidades separadas obligaría a redundancia de datos. El modelo unificado simplifica la persistencia y la consulta a través de endpoints consolidados (/app-state y /api/v1/feed).

5. Estado de Avance y Resultados Operativos

Durante el último ciclo de desarrollo, el equipo ha alcanzado los siguientes hitos verificables en el repositorio:

5.1 Infraestructura y Backend

- **Base de Datos PostgreSQL Conectada:** Base de datos relacional PostgreSQL plenamente operativa en contenedor Docker, con esquemas de tablas sincronizados y versionados con Alembic.
- **API REST Modular Implementada:** Conjunto completo de endpoints REST organizados en routers independientes (/auth, /users, /feed, /events, /news, /classroom, /media), con serialización y validación estricta en Pydantic.
- **Documentación Interactiva OpenAPI:** Especificación OpenAPI 3.0 funcional y accesible en /docs, permitiendo la auditoría y prueba funcional directa de cada endpoint.
- **Despliegue y Conexión de Servidor con NetBird:** Servidor de staging implementado sobre infraestructura contenerizada y accesible de forma segura mediante la red Zero-Trust de NetBird para pruebas de integración.

5.2 Integraciones Externas Validadas

- **Integración del Feed de Instagram:** Conector operativo con SociableKIT para la agregación automática de publicaciones institucionales dentro del feed unificado.
- **Gestión de Identidad y Roles con Google Workspace:**
    - Flujo de inicio de sesión con Google OAuth 2.0 restringido al dominio @upa.edu.py.
    - Endpoint seguro de desarrollo (dev-login) con emisión de tokens JWT para pruebas unitarias y de integración.
    - Detección y mapeo de grupos de Google para asignación automática de carrera y semestre.
- **Sincronización de Google Classroom:** Conexión con la API de Classroom para la lectura de cursos y extracción de fechas de entrega de tareas hacia la agenda del estudiante.

5.3 Funciones Principales Operativas

- **Gestión Unificada de Noticias y Eventos:** Endpoints CRUD para la creación, categorización (MELDUNG, ANKUENDIGUNG), expiración de avisos y fijado de comunicados urgentes.
- **Gestión de Asistencia a Eventos (RSVP):** Tabla event_registrations funcional para registrar asistencia o baja de eventos con restricción de unicidad por usuario.
- **Control de Acceso Basado en Roles (RBAC):** Blindaje de rutas administrativas; los usuarios estándar tienen acceso de lectura e inscripción, mientras que las operaciones de publicación, carga de afiches y auditoría están reservadas para usuarios con rol admin.
- **Servicio de Carga Multimedia:** Endpoint /api/media/upload habilitado para almacenar afiches universitarios, sirviéndolos de manera estática y segura desde /uploads.

5.4 Frontend y Herramientas Administrativas

- **Dashboard de Pruebas:** Interfaz provisional para monitorear en tiempo real la salud de la API, validar respuestas JSON y verificar la sincronización de estados del sistema (/app-state).
- **Cockpit Editorial Web-Admin:** Prototipo funcional del portal web de administración que permite al personal de redacción crear publicaciones, programar eventos en el calendario, fijar banners de emergencia y subir imágenes de afiches.
- **App Nativa Android en Proceso de Conexión:** Estructura modular en Android Studio (Kotlin / Jetpack Compose) con pantallas base de Inicio, Agenda y Perfil preparadas para el consumo de la API REST.